

# COMP1324 Algorithmic Approximation Algorithms

Hikmat Farhat

# A Little Complexity Theory

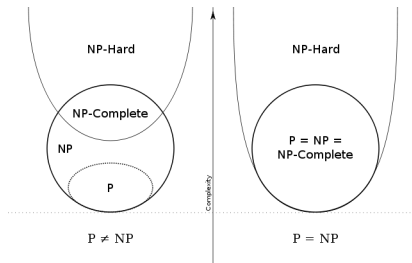
## NP-Completeness in short.

- **P** is a complexity class that contains all decision problems (yes/no) that can be **solved** in polynomial time
- **NP** is a class of decision problems that we can **verify** true instances in polynomial time.
- Clearly  $P \subseteq NP$
- Is  $P = NP$ ? or  $P \subset NP$ ? **Millennium Prize Problem**
- Polynomial time reduction: a decision problem  $B$  is said to polynomial time reducible to decision problem  $A$ ,  $B \leq_p A$  iff  $\exists$  a map  $\sigma$ , computable in polynomial time, such that:

$$x \in B \Leftrightarrow \sigma(x) \in A$$

# A Little Complexity Theory

- A problem  $A$  is **NP-Complete** if
  - 1  $A$  is in **NP**, and
  - 2  $\forall B \in NP$  we have  $B \leq_p A$  ( $A$  is **NP-Hard**)
- **NP-Complete** problems are the 'hardest' in **NP**.



- if  $\exists A$  NP-complete **and**  $A \in P$  then  $P = NP$
- Because of the above, most computer scientists **believe** that ( $P \neq NP$ ).

- Since many optimization problems are NP-hard (technically, NP-hard refers to the corresponding decision problems), we cannot hope to find an efficient algorithm (less than  $O(2^n)$ ) that solves them exactly.
- Instead, we must settle for a **good** approximate solutions.
- How do we measure the quality of an approximate solution?
- We look for algorithms that finds a solution to an optimization problem that is guaranteed to be within a certain factor of the optimal solution.

### Definition ( $\alpha$ -approximation algorithm)

An algorithm is an  $\alpha$ -approximation algorithm if it returns a solution that is within a factor  $\alpha$  of the optimal solution.

# Approximation Algorithms

## Example

Let  $OPT(I)$  be the optimal solution for instance  $I$  of a minimisation problem. An algorithm  $A$  is said to be an  $\alpha$ -approximation if for all instances  $I$  we have:

$$OPT(I) \leq A(I) \leq \alpha \cdot OPT(I), \alpha > 1$$

- What would be the definition of an  $\alpha$ -approximation algorithm for a maximisation problem?

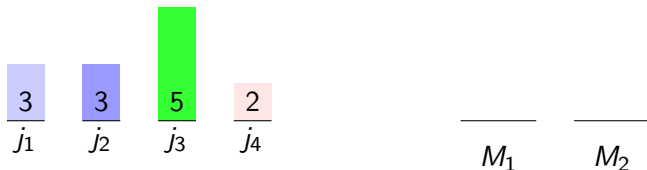
# Example: Load Balancing

- We are given  $n$  tasks with processing times  $t_1, t_2, \dots, t_n$ .
- We want to distribute the tasks to run on the  $m$  machines such that the load is balanced.
- The load of a machine is the sum of the processing times of the tasks assigned to it.
- The goal is to minimize the **makespan**, i.e. the maximum load of any machine.
- Let  $L(M_i)$  be the load of machine  $M_i$ ; then the makespan is:

$$\max_{1 \leq i \leq m} L(M_i)$$

# Example: Load Balancing

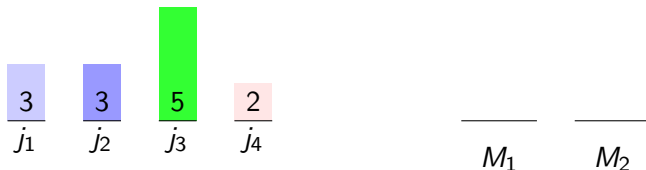
- Consider the following instance of the load balancing problem:
- There are four tasks, with  $t_1 = 3$ ,  $t_2 = 3$ ,  $t_3 = 5$ ,  $t_4 = 2$  and two machines,  $m = 2$ .
- An illustration of the instance is shown below.



- What is the optimal makespan?

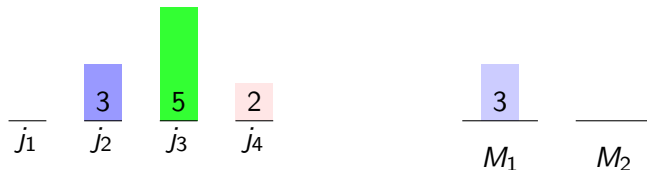
# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



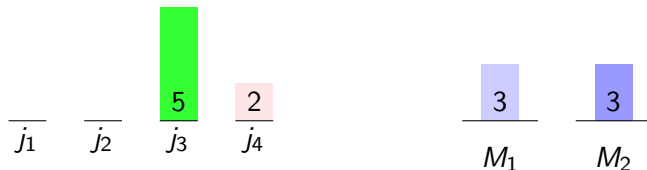
# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



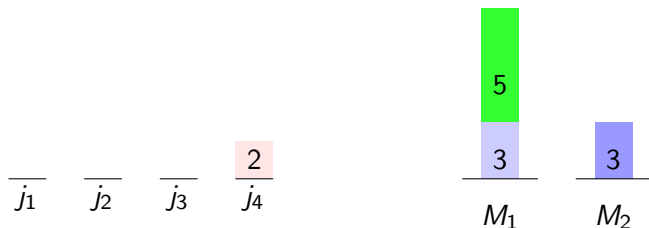
# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



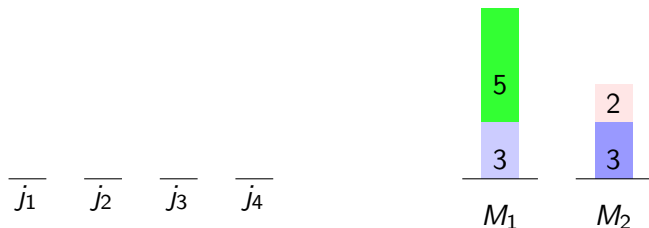
# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



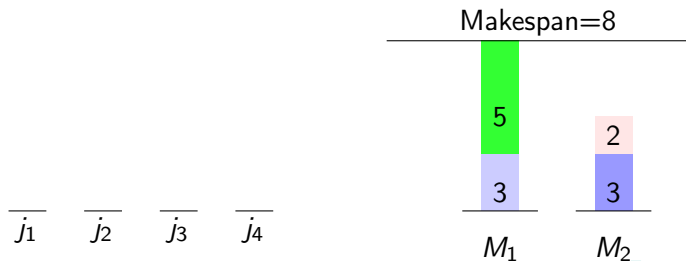
# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



# Example: greedy algorithm

- The minimum makespan problem is NP-hard.
- We will look for an  $\alpha$ -approximation algorithm.
- **Greedy algorithm:** consider tasks in order and assign each task to the machine with the current smallest load.
- The algorithm is illustrated on the instance below.



# Analysis of the greedy algorithm

- How good is the greedy algorithm?
- We are looking for an  $\alpha$ -approximation algorithm.
- In other words let  $G(I)$  be the makespan of the greedy algorithm for instance  $I$  and  $OPT(I)$  the optimal makespan.
- We want to find  $\alpha$  such that:

$$OPT(I) \leq G(I) \leq \alpha \cdot OPT(I)$$

- But if we can compute  $OPT(I)$  we would not need the greedy algorithm !
- Instead, we will look for a lower bound for  $OPT(I)$
- How does finding a lower bound for  $OPT(I)$  help us?

# Finding a lower bound

- Let  $L(M)$  be the load of machine  $M$  then the average load is:

$$\frac{1}{m} \sum_{i=1}^m L(M_i) = \frac{1}{m} \sum_{j=1}^n t_j$$

- We deduce that there exists a machine  $M_i$  such that:

$$L(M_i) \geq \frac{1}{m} \sum_{k=1}^n t_k$$

- Also, the largest task must be assigned to some machine  $M_j$ :

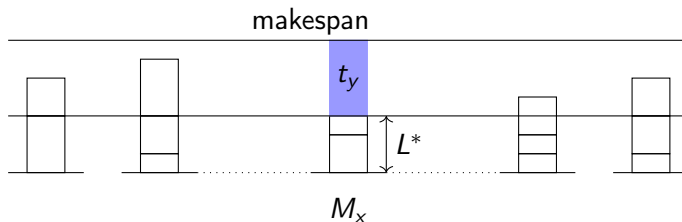
$$\exists M_j, L(M_j) \geq \max_{1 \leq k \leq n} t_k$$

- Combining the two inequalities we have:

$$OPT(I) \geq \max \left\{ \frac{1}{m} \sum_{j=1}^n t_j, \max_{1 \leq i \leq n} t_i \right\}$$

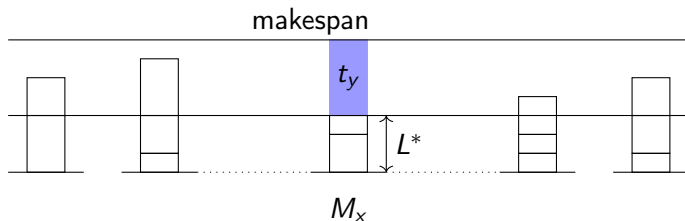
# Analysis of the greedy algorithm

- Let  $M_x$  be the machine with the largest load when greedy terminates.
- Let  $J_y$  be the **last task** assigned to  $M_x$  with processing time  $t_y$ .
- Let  $L^*$  the load of  $M_x$  **before** it was assigned  $J_y$ .
- Note** that  $J_y$  is not necessarily the last task assigned by the greedy algorithm, i.e.  $1 \leq y \leq n$ .



# Analysis of the greedy algorithm

- From the previous definitions we have  $makespan = L^* + t_y$ .
- Since  $J_y$  was assigned to  $M_x$  then  $M_x$  had the **smallest load** when  $J_y$  was assigned.
- The above implies that  $L^* \leq \frac{1}{m} \sum_{j=1}^{y-1} t_j \leq \frac{1}{m} \sum_{j=1}^n t_j$



# Analysis of the greedy algorithm

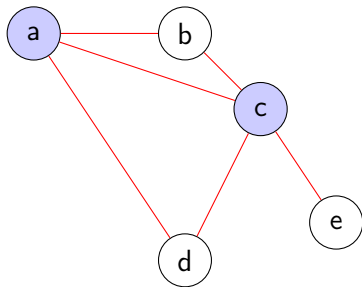
- The greedy makespan is determined by the load of machine  $M_x$ , therefore

$$\begin{aligned}
 G(I) &= L^* + t_y \\
 &\leq \frac{1}{m} \sum_{j=1}^n t_j + \max_{1 \leq i \leq n} t_i \\
 &\leq 2 \cdot \max \left\{ \frac{1}{m} \sum_{j=1}^n t_j, \max_{1 \leq i \leq n} t_i \right\} \\
 &\leq 2 \cdot OPT(I)
 \end{aligned}$$

- Therefore the greedy algorithm is a 2-approximation algorithm.

# Example: Vertex cover

- Given a graph  $G = (V, E)$  a **vertex cover** is a set  $C \subseteq V$  such that for every edge  $(u, v) \in E$  either  $u \in C$  or  $v \in C$ .
- Our goal is to find a minimum vertex cover, i.e. a vertex cover with the smallest possible number of vertices.
- Below is an example of a minimum vertex cover  $C = \{a, c\}$ .



# Vertex cover: approximation algorithm

- Below is a simple approximation algorithm for the vertex cover problem.

---

**Algorithm 1:** Appr. algorithm for minimum vertex cover.

---

**input:** A graph  $G = (V, E)$

**Result:** Returns a vertex cover of  $G$

**function**  $VC(G)$

$C \leftarrow \emptyset$

$E_c \leftarrow E$

**while**  $E_c \neq \emptyset$  **do**

select edge  $(u, v) \in E_c$

$C \leftarrow C \cup \{u\}$

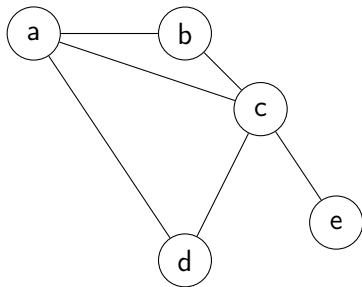
remove all edges incident to  $u$  from  $E_c$

**return**  $C$

---

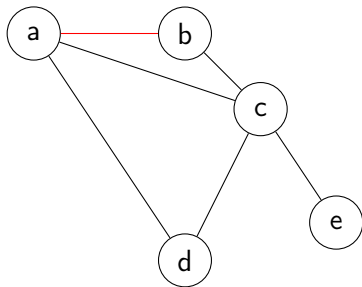
# Example of greedy algorithm for vertex cover

- Let us apply the algorithm to the graph below and assume the edges are processed in alphabetical order  
 $E_c = \{(a, b), (a, c), (a, d), (b, c), (c, d), (c, e)\}$ .



# Example of greedy algorithm for vertex cover

- Let us apply the algorithm to the graph below and assume the edges are processed in alphabetical order  
 $E_c = \{(a, b), (a, c), (a, d), (b, c), (c, d), (c, e)\}$ .

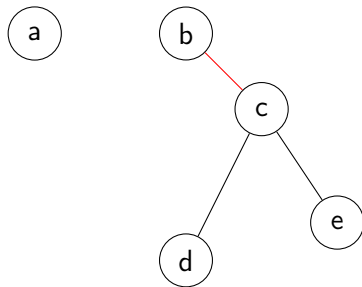


$$C = \{a\}$$

# Example of greedy algorithm for vertex cover

- Let us apply the algorithm to the graph below and assume the edges are processed in alphabetical order

$$E_c = \{(a, b), (a, c), (a, d), (b, c), (c, d), (c, e)\}.$$

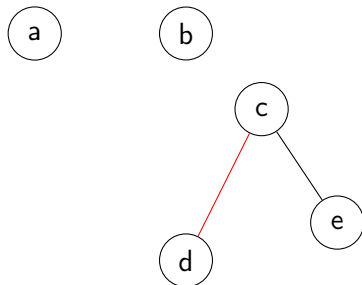


$$C = \{a, b\}$$

# Example of greedy algorithm for vertex cover

- Let us apply the algorithm to the graph below and assume the edges are processed in alphabetical order

$$E_c = \{(a, b), (a, c), (a, d), (b, c), (c, d), (c, e)\}.$$

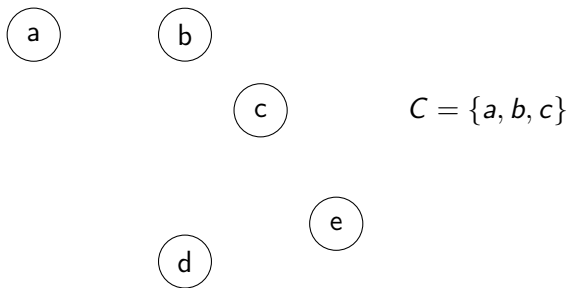


$$C = \{a, b, c\}$$

# Example of greedy algorithm for vertex cover

- Let us apply the algorithm to the graph below and assume the edges are processed in alphabetical order

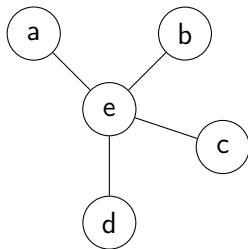
$$E_c = \{(a, b), (a, c), (a, d), (b, c), (c, d), (c, e)\}.$$



Obviously the solution depends on the order of processing of edges.  
Note that the optimal solution is  $C = \{a, c\}$ .

# Worst case

- While the previous algorithms looks reasonable, in some cases it can be arbitrarily bad.
- For example in the graph below if the edges are processed in alphabetical order the size of the VC is  $|V| - 1$ .
- Whereas the minimum VC has size 1, i.e.  $C = \{e\}$



# Modified algorithm

- It turns out we can get a 2-approximation algorithm by modifying the previous algorithm.

---

**Algorithm 2:** Modified Appr. algorithm for minimum VC.

---

**input:** A graph  $G = (V, E)$

**Result:** Returns a vertex cover of  $G$

**function** VC( $G$ )

$C \leftarrow \emptyset$

$E_c \leftarrow E$

**while**  $E_c \neq \emptyset$  **do**

select edge  $(u, v) \in E_c$

$C \leftarrow C \cup \{u, v\}$

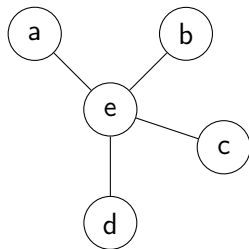
remove all edges incident to  $u$  and  $v$  from  $E_c$

**return**  $C$

---

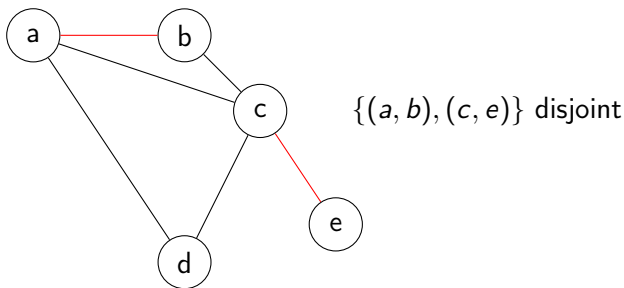
# Example

- If we apply the modified algorithm to the graph below, processing the edges in alphabetical order, as before, we get  $C = \{a, e\}$  which is twice the size of the minimum.
- It turns out the modified algorithm is a 2-approximation algorithm.



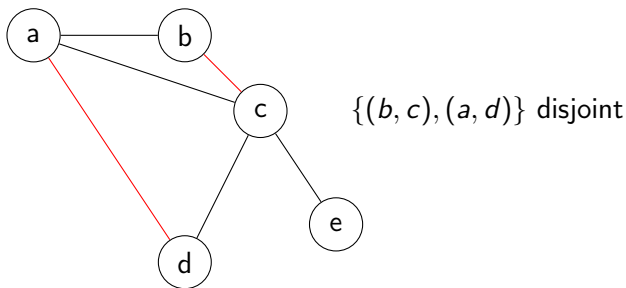
# Disjoint edges

- As we did for load balancing we look for a lower bound of the optimal solution.
- To find a lower bound we need the concept of **disjoint edges**.



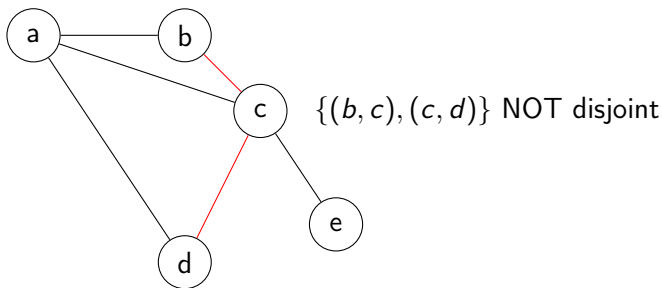
# Disjoint edges

- As we did for load balancing we look for a lower bound of the optimal solution.
- To find a lower bound we need the concept of **disjoint edges**.



# Disjoint edges

- As we did for load balancing we look for a lower bound of the optimal solution.
- To find a lower bound we need the concept of **disjoint edges**.

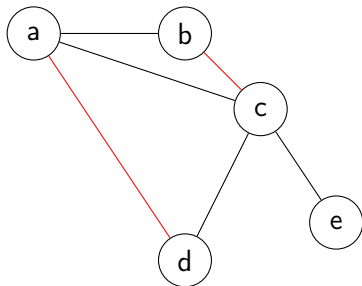


# Lower bound

- Let  $OPT(G)$  be the size of the minimal vertex cover for instance  $G$ .
- For any set  $E^* \subseteq E$  of pairwise disjoint edges we have:

$$OPT(G) \geq |E^*|$$

- This is true because **at least one** vertex is needed to cover each edge in  $E^*$ . Below is an example.



# Modified algorithm

- Because of line 4 in the algorithm, the edges processed by the algorithm on line 2 are pairwise disjoint.
- Since  $|C|$  is twice the number of processed edges then  $|C| \leq 2 \cdot \text{OPT}(G)$

---

**Algorithm 3:** Modified Appr. algorithm for minimum VC.

---

**input:** A graph  $G = (V, E)$

**Result:** Returns a vertex cover of  $G$

**function** VC( $G$ )

```

     $C \leftarrow \emptyset$ 
     $E_c \leftarrow E$ 
1  while  $E_c \neq \emptyset$  do
2      select edge  $(u, v) \in E_c$ 
3       $C \leftarrow C \cup \{u, v\}$ 
4      remove all edges incident to  $u$  and  $v$  from  $E_c$ 
5  return  $C$ 
```

# References

- Approximation algorithms are covered in chapter 35 of your [textbook](#)
- A different set of approximation algorithms are covered in chapter 20 the Algorithms Illuminated [book](#)
- An extensive treatment of approximation algorithms can be found in the [book](#) by Williamson and Shmoys.